

Part II: KD-Tree & QuadTree for k-NN Search

Implementation

KD-Tree (source/kd_tree/kd_tree.hh) — recursive binary tree, header-only C++20. Each node stores the split axis (highest variance dimension), split value (median), and bounding box lo/hi. Build selects the pivot by `nth_element` at the median index. Query uses a max-heap of size `k`; a subtree is pruned when the distance from the query point to the node's bounding box exceeds the current heap worst.

QuadTree (source/quad_tree/quad_tree.hh) — 2D-only recursive structure, same interface. Each node is split into four equal quadrants at the geometric midpoint; cells with ≤ 8 points or depth ≥ 32 become leaves. Query pruning is the same box-distance rule applied to each quadrant's bounding box; children are visited in order of increasing box distance.

Both implement `NNSIndex` (source/generic/index.hh):

```
virtual void build(PointsPtr aPoints) = 0;
virtual QueryResult query(const Point &aQ, size_t aK) const = 0;
```

Logger (source/generic/logger.hh) — compile-time-switched event emitter (`ENABLE_LOGGING` CMake flag). Emits structured tokens (SPLIT, QUAD, PRUNE, PRUNEBOX, VISIT, ACCEPT, EVICT, RESULT) consumed by the visualiser. Zero overhead in release builds (if `constexpr`).

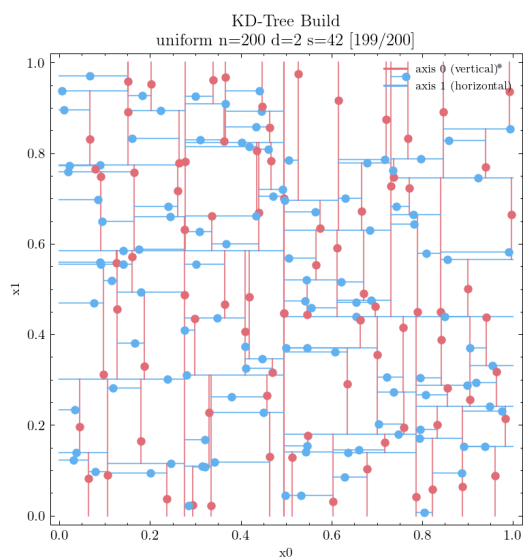
Benchmark (source/benchmark/) — measures build time, mean/std query latency, and mean/std nodes visited over configurable iterations. Stats accumulate per (algo, distribution) in CSV files. `sweep.sh` automates data generation, correctness check (neighbour index comparison via `diff`), and two experiments.

Visualiser (source/visualize.py) — `matplotlib.FuncAnimation` driven by event logs. Renders KD-Tree axis-split lines, QuadTree midpoint crosses (depth-coloured), and query animations (visited/accepted/pruned points, expanding search radius).

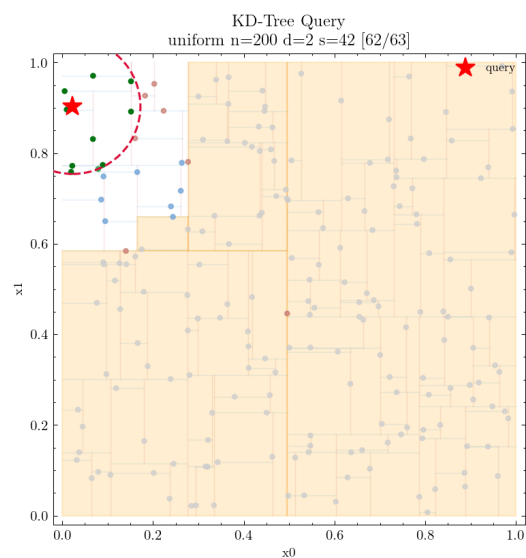
Visualisation

Final frames of build and query animations on uniform $n=200$, $d=2$.

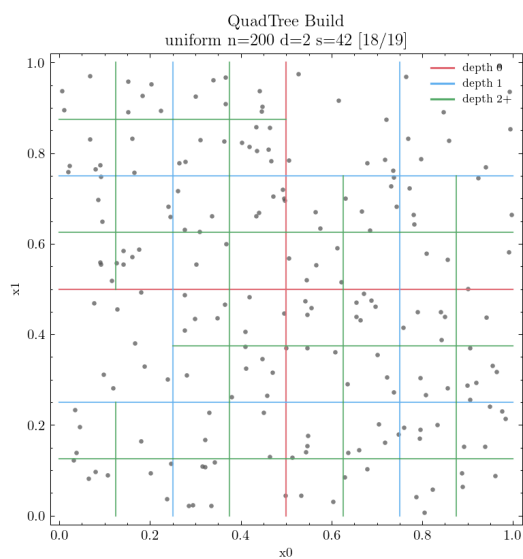
KD-Tree build



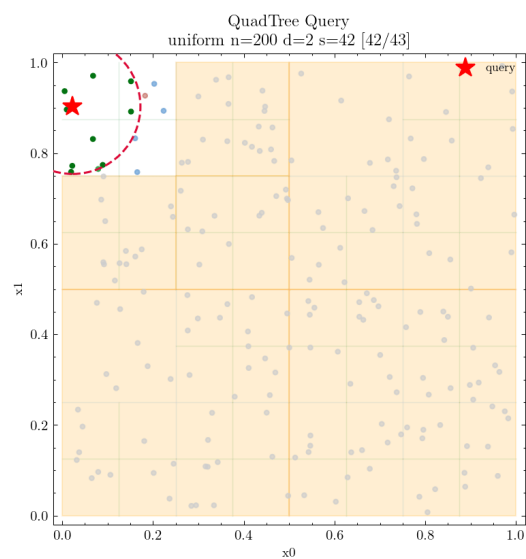
KD-Tree query



QuadTree build



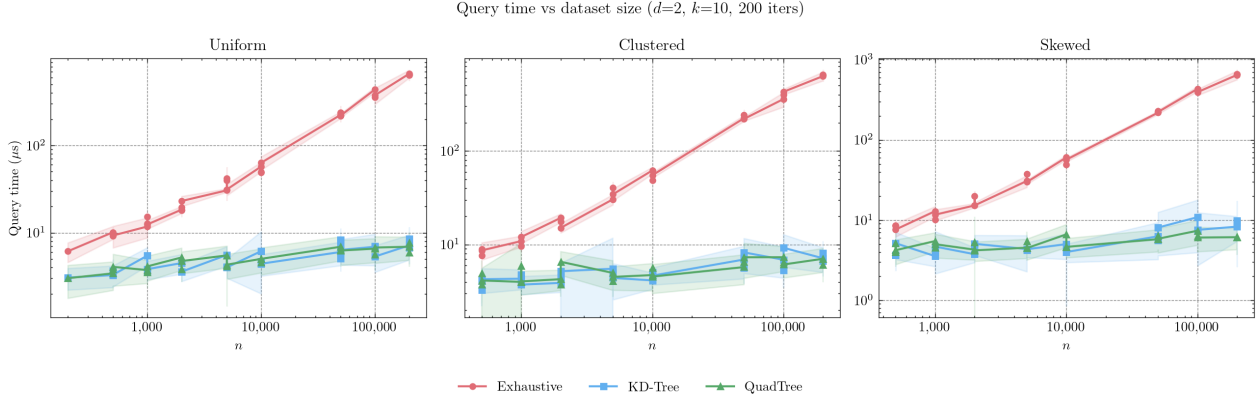
QuadTree query



Benchmark Results

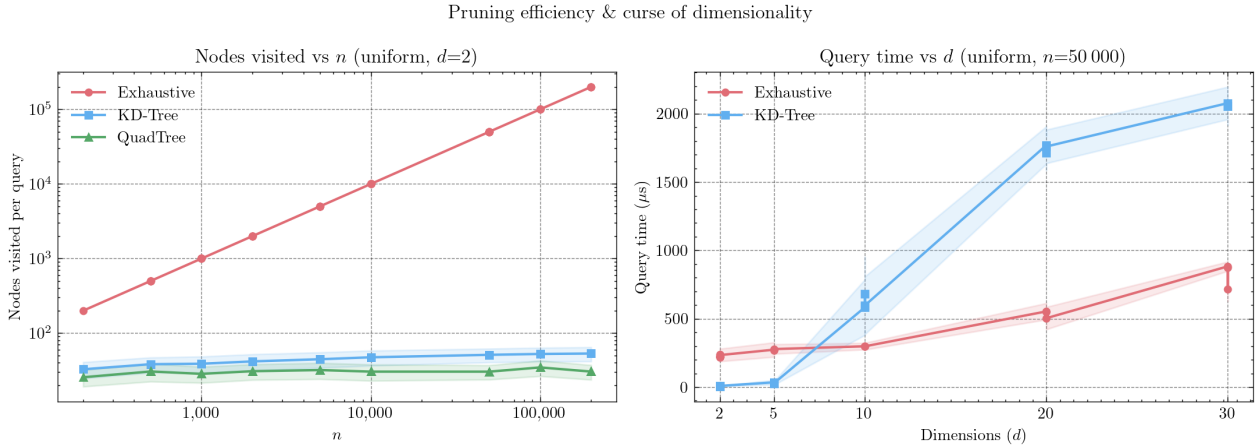
All runs: k=10, 200 iterations, seed=42.

Query latency vs n — d=2



n	Exhaustive	KD-Tree	QuadTree	KD speedup	Quad speedup
1 000	~13 μ s	~4 μ s	~4 μ s	3 $\times$	3 $\times$
10 000	~52 μ s	~5 μ s	~5 μ s	10 $\times$	10 $\times$
100 000	~390 μ s	~7 μ s	~7 μ s	56 $\times$	56 $\times$
200 000	~648 μ s	~7.5 μ s	~7.0 μ s	86$\times$	93$\times$

Pruning efficiency & curse of dimensionality



Nodes visited at $n=200\,000$, $d=2$, uniform:

Algorithm	Nodes visited	% of dataset
Exhaustive	200 000	100%
KD-Tree	~53	0.03%
QuadTree	~31	0.02%

Effect of distribution — n=200 000, d=2

Distribution	Exhaustive	KD-Tree	QuadTree
Uniform	~645 μ s	~7.5 μ s	~7.0 μ s
Clustered	~643 μ s	~7.3 μ s	~6.7 μ s
Skewed	~652 μ s	~9.1 μ s	~6.1 μ s

KD-Tree degrades slightly on skewed data (unbalanced median splits). QuadTree is unaffected — the dense corner gets isolated quickly regardless.

KD-Tree: curse of dimensionality — uniform, n=50 000

d	Exhaustive	KD-Tree	KD speedup
2	~220 μ s	~7 μ s	31 $\times$
5	~275 μ s	~30 μ s	9 $\times$
10	~298 μ s	~620 μ s	0.5 $\times$
20	~535 μ s	~1 750 μ s	0.3 $\times$
30	~824 μ s	~2 060 μ s	0.4 $\times$

KD-Tree breaks even with exhaustive around d=8-10; at d=20+ it visits essentially the entire dataset and becomes strictly slower.